

Introduction

The stock market is a notoriously unpredictable and chaotic environment. For this reason, many mathematical models are employed to attempt to evaluate levels of risk and gauge expectations about the future. In particular, options contracts utilize the Black-Scholes model to set prices (European options contracts, American contracts use a different model).

Overview of Options

Options contracts are contracts that grant the owner permission to buy or sell a stock at a specific price at a specific time. For our report we have analyzed call options specifically, where contracts grant the right to buy a stock. To provide an example, one might buy a call option contract for \$30 that grants them the right to buy shares of a stock at \$200 at a future date. If the price of the stock on that given date is over \$200 the owner will execute their contract, buying shares and instantly selling them for a profit (which may or may not cover the original cost of the contract). Otherwise, they will simply let the contract expire and their loss will be the original cost of it.

The Black-Scholes Model

The Black-Scholes model uses a concept from the field of Physics called Brownian motion to emulate stock price movements. Essentially, a random walk of stock price movements is taken, where stock prices move randomly in a certain direction over daily increments. This randomness comes from a standard normal distribution, and the specific price movements are based on past price dynamics of the given stock being evaluated. Instead of measuring these random walks one by one, Black-Scholes uses differential equations to solve for the expected value of all these walks mathematically. The final prices at the end of each walk can be thought of as being averaged to arrive at a predicted final price for a stock, with all possible price paths being accounted for.

With this, the price of a call option contract is set to be the expected value of the stock minus the price at which it can be bought as specified by the contract (called the strike price), both discounted to their present value at the time of purchase. To discount to present value is to account for the fact that money today can be invested for a future return, so it is essentially more valuable. Discounting adjusts future dollars to what their value would be today. The price of a contract ultimately represents the exact expected value of holding the contract, set efficiently such that no one party can instantly profit from owning it.

$$C = SN(d_1) - Ke^{-rT}N(d_2)$$

A formulaic representation, where C represents the price of the option, the first term represents the expected value, and the second term represents the strike price.

The Monte Carlo Approach

While the Black-Scholes model is accepted as the industry standard for European options pricing, there are other possible methods. Where Black-Scholes takes a theoretical expectation of all possible price paths, these price paths can also be manually simulated. In theory, these simulations would converge to the Black-Scholes calculated price with a sufficiently large number of samples. Our approach involves running these simulations and observing the convergence to the Black-Scholes model.

Black Swan Events

One of the flaws of the many mathematical models and theoretical approaches to predicting future performance in the stock market is the inability to respond to real-world events. Because large-scale geopolitical or social events are inherently chaotic and unpredictable, they are rarely reflected in the models, which rather rely on underlying patterns of the market. To explore this potential deficiency, we can adapt our model to incorporate Black Swan events: highly rare and spontaneous real-world events that have major market implications. Black Swan events in history have included the COVID pandemic, the 2008 financial crisis, and the early 2000s dot com bubble, all of which resulted in major price changes across the stock market.

Model

Setup

Our basic setup involves importing past financial data from Yahoo Finance to compile key metrics on price and past volatility of our chosen stocks. We will then simulate price movements using random walks, averaging all of them to arrive at a predicted stock price and predicting option contract payoff, which will then be identical to the price of the contract. All of the code for the model will be written in python.

The Code

```
def price_monte_carlo_call(ticker, K=None, t=1.0, r=0.05, n_sims=100, n_steps=252):  
  
    stock_data = yf.download(ticker, period="2y", progress=False)  
    stock_data['Log_Returns'] = np.log(stock_data['close'] / stock_data['close'].shift(1))  
    stock_data = stock_data.dropna()  
  
    P_0 = stock_data['close'].iloc[-1]  
    daily_volatility = stock_data['Log_Returns'].std()  
  
    sigma = daily_volatility * np.sqrt(252)  
  
    dt = t/n_steps
```

The core of the model is a function defined above. The parameters are:

- Ticker: the stock ticker of the desired stock to simulate
- K: The strike price, which by default will be the price of the stock at the time the simulation is run.
- t: the duration of the options contract, i.e. the time from the present after which the contract can be executed.
- r: the risk-free rate. This is defined generally in economics as the rate of return for which there is no risk (e.g. a government bond). It will be used to create price paths as well as discount prices to their present value.
- n_sims: the number of simulations ran.
- n_steps: the number of increments for which a random price movement is generated.

To create our function, we start by downloading previous price data for the given ticker going back 2 years from the time the function is used. Log returns of the prices are then obtained from this data, which represent the percentage change in price of a stock from one day to the next. Daily volatility is obtained by taking the standard deviation of these log returns. This represents the overall volatility and variation of the stock price movements. In the Black-Scholes model, sigma is used as shown above to represent variation when modeling price paths, and we will proceed similarly.

```
S = np.zeros((n_steps + 1, n_sims))
S[0] = P_0

for t_step in range(1, n_steps + 1):
    z = np.random.standard_normal(n_sims)

    drift = (r - 0.5 * sigma ** 2) * dt
    shock = sigma * np.sqrt(dt) * z

    S[t_step] = S[t_step - 1] * np.exp(drift + shock)

final_prices = S[-1]

if K is None:
    K = P_0.item()
```

We then create an array with the rows representing each trading day of an individual price path and the columns representing each distinct price path. The first day for all simulations will have the original price of the stock (P_0). To generate the random component of our random walk, we use a standard normal distribution and assign a z value to each step of each simulation.

$$P_t = P_{t-1} \cdot e^{(r - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z}$$

Our model uses the above equation to calculate the price paths for each simulation, where P_t is the price of the stock on one given day and Z represents the random Z value. The other variables are the same as defined earlier. The term $(r - 0.5 * \sigma^2)$ is called the drift, and represents the overall underlying trend of the stock over time, which we define as the risk free rate minus an uncertainty term using sigma to account for the volatility of the stock. The second term in the exponent is the shock, where our random z value represents the random movement and sigma represents the magnitude of that movement based on the individual stocks history. This is the random variation in the stock from day to day which drives our random walk. Both of these components are defined in the code and reflected in the formula one line below. The calculated prices from this formula fill the array, with the final row giving our final prices.

```
payoffs = np.maximum(final_prices - K, 0)
expected_payoff = np.mean(payoffs)
discount_factor = np.exp(-r*t)
monte_carlo_call_price = expected_payoff * discount_factor

return{
    "Ticker": ticker,
    "Call Price": round(float(monte_carlo_call_price), 2),
    "Strike Price": round(K, 2)
}
```

Using the final prices, we calculate the expected payoff of the contract. If the final price is greater than the strike price, their difference is the payoff. If the final price is below the strike price the payoff is 0, as the contract owner will simply let the contract expire. Taking the average of these payoffs and discounting to the present gives us our final call option price.

Black Swan Events

To create a new model incorporating black swan events we will define a new function identical to the original but with some slight modifications. The core premise of this new model is to generate price paths in the same way as before, but add a new term to the formula called “Swing”. Swing has a value of 1 most of the time, but occasionally will alter the price change on a given day of the simulation. The magnitude and frequency of a “Swing” can be modified as parameters in our new function, with defaults being 1 swing per year and randomly either a 10% increase or decrease in price for each swing. Swing reflects black swan events because they theoretically happen as complete shocks to the market with no prior knowledge and have major price implications.

$$P_t = P_{t-1} \cdot e^{(r - \frac{1}{2}\sigma^2)dt + \sigma\sqrt{dt}Z} \cdot \text{Swing}$$

```
swing_occurs = np.random.rand(n_sims) < freq
swing_multiplier = np.ones(n_sims)

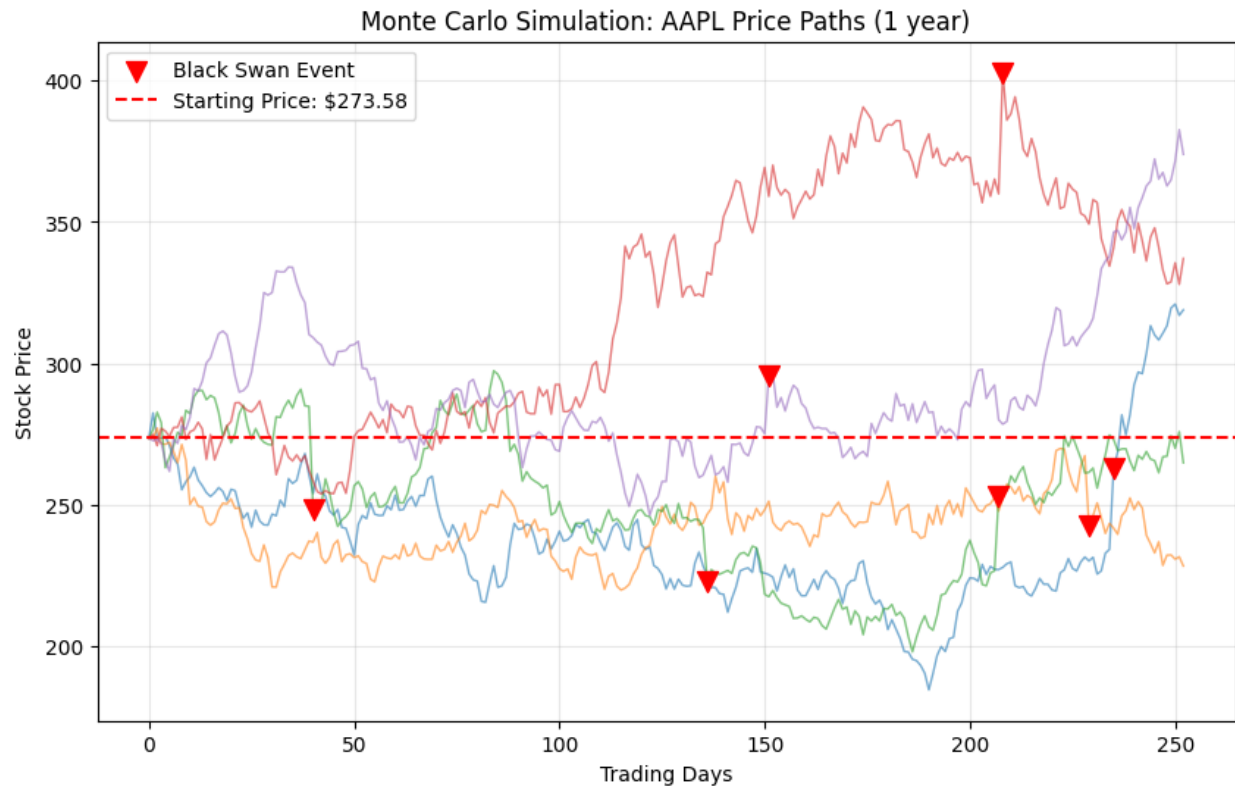
num_swings = np.sum(swing_occurs)
if num_swings > 0:
    directions = rng.choice([-1,1], size=num_swings)
    swing_multiplier[swing_occurs] = 1 + delta * directions
```

freq and delta are new parameters in our function representing, respectively, the probability on a given day of a swing occurring and the magnitude of that swing (e.g. 0.1 for 10%).

To accomplish this in the code, we use random number generation and compare it to our predefined frequency of a swing occurring. With this we will have a list of days in each simulation for which a black swan event will occur. We randomly decide whether that swing will be positive or negative and encode it as our swing multiplier (e.g. 0.9 for a 10% decrease in price and 1.1 for a 10% increase, with most values being 1 for no swing occurring).

```
S[t_step] = S[t_step-1] * np.exp(drift + shock) * swing_multiplier
```

Finally, we duplicate the formula from our original function and multiply it by the swing multiplier to generate an array of price paths that now includes black swan events. Final prices and payoffs are calculated identically to the original function.



An example of price paths with black swan events.

Recreating the Black-Scholes Model

To draw comparison to our Monte Carlo simulations, we will recreate the actual Black-Scholes model in code.

```
def extract_data(data,ticker,t):
    close_data = data["Close"]
    if isinstance(close_data, pd.DataFrame):
        prices = close_data[ticker]
    else:
        prices = close_data
    log_returns = np.log(prices / prices.shift(1)).dropna()
    day_std = log_returns.std()
    o = float(day_std) * np.sqrt(252)
    S = float(prices.iloc[-1])
    return S,o
```

We start by extracting data from Yahoo Finance, including log returns and daily volatility.

```
def make_d1_d2(S,K,r,o,t):
    d1 = (math.log(S/K) + (r + (o**2)/2) * t)/(o * math.sqrt(t))
    d2 = d1 - o * math.sqrt(t)
    return d1, d2
```

We then define the key components of the differential equation used in the Black-Scholes model. These components are written in terms of the variables described earlier; price, strike price, risk free rate, daily volatility, and time until expiration of the contract (price is now S and volatility or sigma is now o).

```
def black_scholes_call(S,K,r,t,d1,d2):
    Nd1 = norm.cdf(d1)
    Nd2 = norm.cdf(d2)
    C = S * Nd1 - K * np.exp(-r * t) * Nd2
    return C
```

Using these components, d1 and d2, as well as our other variables we can plug into the final Black-Scholes equation (shown below) to arrive at our call price.

$$C = SN(d_1) - Ke^{-rT}N(d_2)$$

```
def blackscholes(ticker,K,t,r,call_put,start,end):
    data = yf.download(ticker, start=start, end=end, auto_adjust=True)
    S,o = extract_data(data,ticker,t)
    d1,d2 = make_d1_d2(S,K,r,o,t)
    if call_put == "call":
        option_price = black_scholes_call(S,K,r,t,d1,d2)
    elif call_put == "put":
        option_price = black_scholes_put(S,K,r,t,d1,d2)
    else:
        return "Must enter call or put"
    return f"{ticker}: {call_put} option price is {option_price}"
```

The final function for Black-Scholes option pricing. Parameters are the same as before, with call_put denoting the type of contract (which for us is a call), and start and end denoting the timeframe of the contract.